GUIA TÉCNICO | 10

Hiperparâmetros, inicialização e treinamento

Capítulo 10/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Diferenciar parâmetros treináveis de hiperparâmetros e estudar escala das inicializações.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

Arquitetura, tamanho do batch, taxa de aprendizado, inicialização e número de épocas são hiperparâmetros. Inicialização inadequada pode saturar ativações ou provocar variâncias instáveis. Manter orçamento de busca e validação explícitos.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: Var(W_He)≈2/fan_in

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

A busca de hiperparâmetros deve ocorrer somente dentro dos dados de treinamento com validação cruzada reprodutível.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
rng=np.random.default_rng(2)
fan_in=256
w=rng.normal(0,np.sqrt(2/fan_in),size=(fan_in,64))
print('Desvio He esperado=',round(np.sqrt(2/fan_in),4),'observado=',round(w.std(),4))
assert w.shape==(256,64)
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Seleção de hiperparâmetros dentro do treino, teste intocado

A busca de hiperparâmetros deve ocorrer somente dentro dos dados de treinamento com validação cruzada reprodutível.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import balanced_accuracy_score
X,y=make_classification(n_samples=180,n_features=7,n_informative=5,random_state=9)
Xtr,Xte,ytr,yte=train_test_split(X,y,stratify=y,test_size=.25,random_state=9)
pipe=Pipeline([('scaler',StandardScaler()),('mlp',MLPClassifier(max_iter=160,random_state=
    9))])
cv=StratifiedKFold(n_splits=3,shuffle=True,random_state=9)
search=GridSearchCV(pipe,{'mlp__hidden_layer_sizes':[(8,),(12,)],'mlp__alpha':[.001,.1]},c
    v=cv,scoring='balanced_accuracy',n_jobs=1)
search.fit(Xtr,ytr)
print('Melhor configuração:',search.best_params_,'CV:',round(search.best_score_,3))
print('Teste intocado:',round(balanced_accuracy_score(yte,search.predict(Xte)),3))
```

A métrica no conjunto de teste só deve ser calculada após escolher a configuração, sem repetidamente ajustá-la ao teste.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Uma camada com 128 entradas e 32 unidades Dense tem quantos parâmetros?

Resposta: 128×32 pesos + 32 vieses = 4.128 parâmetros.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn. Treinamentos TensorFlow/Keras não foram validados nesta geração editorial.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide